CLimate data Input for OpenIFS (CLIO)

Sándor István Mahó
sandor.maho@helsinki.fi
Clément Bouvier
Oskari Rantala
Victoria Sinclair

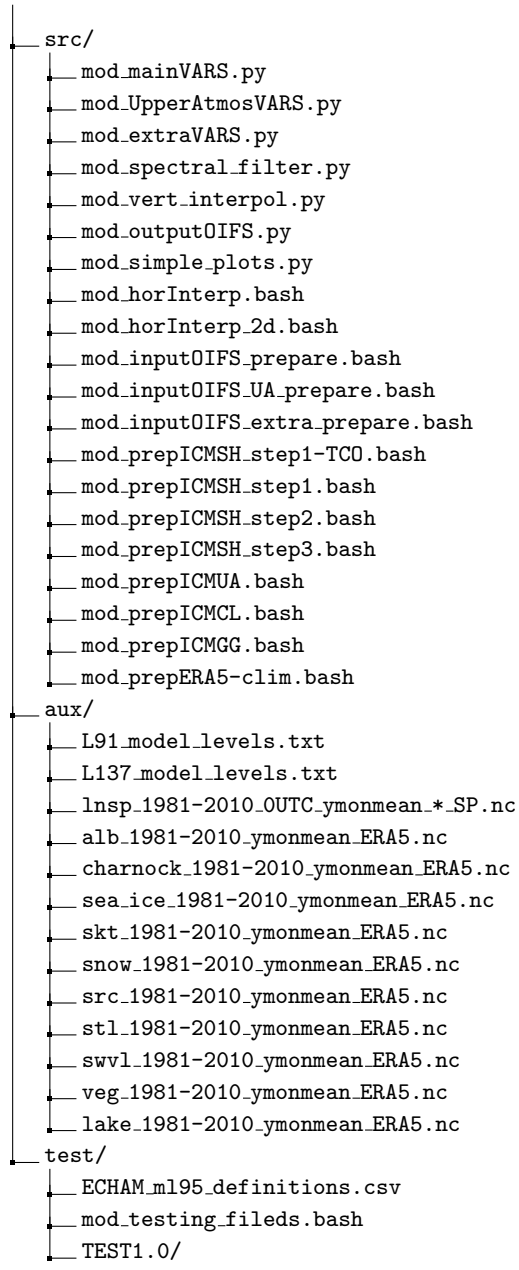
October 15, 2026

1 Description

CLIO rewrites initial condition files for the OpenIFS numerical weather prediction model maintained and developed by ECMWF (https://www.ecmwf.int/en/research/projects/openifs). CLIO is designed to supply meteorological data from CMIP6 models to OpenIFS, thus enabling OpenIFS simulations to be initialised with atmospheric states taken from any climate model (or even any weather and climate model, or reanalysis dataset). Technical details regarding the execution of the CLIO package are given below.

2 Structure of the repository

```
CLIO_release/
├── INPUT_GRIB
│   └── TL399
│       └── ICM* files and ECWAM initial files
├── container/
│   ├── Dockerfile
│   ├── clio-image.simg
│   ├── README.md
│   ├── convert_to_singularity.bash
│   └── To_containerise/
│       ├── cdo-2.4.3.tar.gz
│       ├── eccodes-2.41.0-Source.tar.gz
│       ├── hdf5-1.14.6.tar.gz
│       ├── netcdf-c-4.9.3.tar.gz
│       ├── netcdf-fortran-4.6.2.tar.gz
│       ├── MetviewBundle-2024.4.0-Source.tar.gz
│       └── reqs.python.txt
└── app/
    ├── CLIO_start.py
    ├── CLIO_test.py
    └── plots/
```



As shown in the file tree above, the repository 'CLIO' contains two main directories: *container/* and *app/* . The *container/* directory contains the Singularity image file **clio-image.simg**, which can be used to run CLIO. In addition we provide initial files for one resolution (TL399) inside *INPUT_GRIB*, which are needed for CLIO's testing module.

2.1 Container

The *container/* directory also includes the Dockerfile used to generate the container, as well as the required libraries provided as tar files inside *To_containerise/*. Here, **reqs-python.txt** contains all the necessary python packages, which are added to the container. Furthermore, **README.md** contains instructions on how to generate the container image with **convert_to_singularity.bash**.

2.2 Forecast input files

A publicly available dataset (https://doi.org/10.5281/zenodo.23364583) was created containing all the necessary initial files to run OpenIFS in various resolutions (indicated with their spectral truncation type): TL159, TL255, TL399, TL639, TL1279, TCo319, TCo639 and TCo1279 with 91 model levels. TL1279 and TL639 are also available with 137 model levels. In all of these directories the following files are present:

- **ICMCLxxxxINIT, ICMSHxxxxINIT, ICMGGxxxxINIT, ICMGGxxxxINIUA**: these files contain the surface and atmospheric initial conditions that CLIO will modify.
- **sfcwindin, uwavein, specwavein, wam_grid_tables, wam_subgrid_0, wam_subgrid_1, wam_subgrid_2**: initial conditions for the ocean wave model (ECWAM), which is usually coupled to the atmospheric forecast model, more information can be found under https://confluence.ecmwf.int/display/OIFS/3.4+OpenIFS%3A+Ocean+waves. Note that the ECWAM initial conditions are not modified by CLIO. Their purpose is to have complete initial conditions for an OpenIFS forecast. Note also that the climatological boundary conditions (i.e. greenhouse gas concentrations, aerosol mass data, etc.) as well as specifications for the vertical grid and the reduced Gaussian grid are not provided here, however, the user can obtain them from https://confluence.ecmwf.int/display/OIFS/OpenIFS+Home.
- **wam_namelist**: namelist file for the ECWAM model, which CLIO does not modify.

All of these data was obtained from the OpenIFS Datahub (https://confluence.ecmwf.int/display/OIFS/OpenIFS+Data+Hub). The user is advised to download all the files of the chosen spectral resolution, since CLIO requires OpenIFS initial files that will be updated with the user-defined input (e.g. CMIP6 data). The user is recommended to place the forecast input files of different resolutions into separate directories, ideally inside *INPUT_GRIB*, as it was done for the TL399 resolution, whose initial files are provided.

2.3 Application

The source code and auxiliary data of CLIO can be found in *app/*. The user should use **CLIO_start.py** as the launch script for CLIO in which the namelist can be modified. **CLIO_test.py** may only be used to the test the application after downloading. **CLIO_start.py** launches the main python modules serially: **mod_mainVARS.py**, **mod_extraVARS.py** and **mod_UpperAtmosVARS.py**.

These modules modify the initial condition files starting with ICM, and can be found in *src/*.

2.4 Auxiliary data

Auxiliary data needed for running CLIO is available inside *aux/*. Here the OpenIFS model level definitions (for 91 and 137 levels) are found as csv files. The ERA5 *lnsp* (logarithm of surface pressure) seasonal mean spectral coefficients are also located here for various resolutions named as **lnsp_1981-2010_0UTC_ymonmean_{RES}_SP.nc**, where *RES* denotes the Gaussian grid type. Further 30-year monthly mean climatological files can be found in gridpoint representation on a regular lat-lon grid of 0.25 degrees:

- **alb_1981-2010_ymonmean_ERA5.nc** containing forecast albedo (fal),
- **charnock_1981-2010_ymonmean_ERA5.nc** containing Charnock ($chnk$),
- **lake_1981-2010_ymonmean_ERA5.nc** containing lake mix-layer temperature ($lmlt$), lake mix-layer depth ($lmld$), lake bottom temperature ($lblt$), lake total layer temperature ($ltlt$), lake shape factor ($lshf$), lake ice surface temperature ($lict$) and lake ice total depth ($licd$),
- **sea_ice_1981-2010_ymonmean_ERA5.nc** containing sea ice temperature for layer 1-4 ($istl1$, $istl2$, $istl3$, $istl4$) and sea ice concentration ($siconc$),
- **skt_1981-2010_ymonmean_ERA5.nc** containing skin temperature (skt),
- **snow_1981-2010_ymonmean_ERA5.nc** containing snow albedo (asn), snow depth (sd), temperature of snow layer (tsn) and snow density (rsn),
- **src_1981-2010_ymonmean_ERA5.nc** containing skin reservoir content (src),
- **stl_1981-2010_ymonmean_ERA5.nc** containing soil temperature for layer 1-4 ($stl1$, $stl2$, $stl3$, $stl4$),
- **swvl_1981-2010_ymonmean_ERA5.nc** containing volumetric soil water for layer 1-4 ($swvl1$, $swvl2$, $swvl3$, $swvl4$),
- **veg_1981-2010_ymonmean_ERA5.nc** containing leaf area index for low and high vegetation (lai_lv and lai_hv, respectively).

Note that all of these fields are derived from ERA5 data (doi:10.24381/cds.adbb2d47). Last, in folder *plots/* CLIO saves the figures generated during the sanity checks.

3 Dependencies

To install CLIO from the git repository one requires git-lfs (https://git-lfs.com/), which is an open source Git extension.

The list of required python packages are listed in **reqs-python.txt** file inside *To_containerise/*. Additionally, metview-python (5.22) is also needed to run CLIO. Furthermore, CLIO requires a working installation of CDO (Climate Data Operators, v2.4.3, https://code.mpimet.mpg.de/projects/cdo) and ecCodes (v2.41, https://confluence.ecmwf.int/display/ECC/ecCodes+Home). A Dockerfile with the archive of CDO and ecCodes is given. Moreover, a Singularity container converted from the Docker image is also provided for executing CLIO, which can be found inside *container/*.

4 Workflow

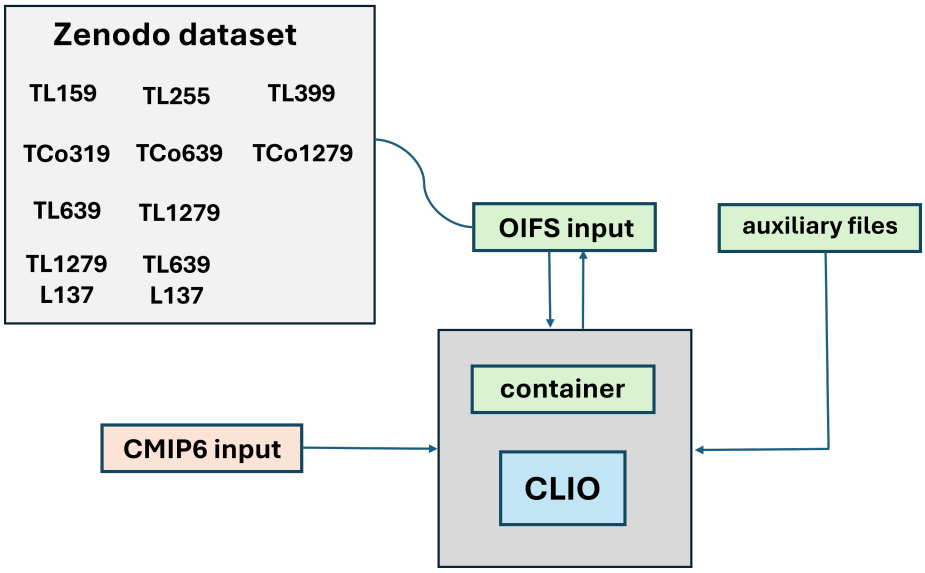


Figure 1: Structure of CLIO: the application is shown with blue background, which together with the container forms the software stack. The OpenIFS (OIFS) input files which CLIO updates are provided at various resolutions as an external data repository. Auxiliary data is also required to run the application, as well as the user-specified CMIP6 input data.

CLIO requires a running environment, which is provided by the container, OpenIFS initial files at various resolutions which will be updated by CLIO, auxiliary files, as well as the user-given CMIP6 input data as can be seen in the flow chart in Figure 1. First, CLIO reads in the CMIP6 input data, then different submodules are called to convert the input into OpenIFS-readable format. This includes regridding the input data both in the horizontal and the vertical, converting CMIP6 units to the ones used by OpenIFS, converting grid point fields to spectral representation and converting NetCDF files to GRIB. Once the GRIB files with CMIP6 data are ready, CLIO updates the OpenIFS initial files using these GRIB files. The auxiliary data is used to update the fields not found in CMIP6 with ERA5 climatologies.

CLIO rewrites four GRIB OpenIFS input files (ICM* files, see https://confluence.

ecmwf.int/display/OIFS/6.2+OpenIFS+Input+Files) with CMIP6 data without changing their metadata. Input data for the ocean wave model - though they are not changed by CLIO - are also provided. External boundary conditions (i.e. aerosols, greenhouse gases, radiation tables) are neither provided, nor updated by CLIO.

CLIO is executed via CLIO_start.py, which launches the three main modules of CLIO serially:

1. mod_mainVARS.py
2. mod_extraVARS.py
3. mod_UpperAtmosVARS.py.

Module **mod_mainVARS.py** rewrites the initial fields in the ICMSHxxxxINIT file (i.e. the prognostic variables in spectral representation): temperature, vorticity, divergence and logarithm of surface pressure. Here and elsewhere, xxxx refers to the experiment name, which is a 4-character code. Module **mod_extraVARS.py** rewrites some fields in the ICMGGxxxxINIT and ICMCLxxxxINIT files, which is controlled by the user (details follow). Module **mod_UpperAtmosVARS.py** rewrites all fields in the ICMGGxxxxINIUA file. Here, the user has some control over the fields that are updated.

The three main modules call further python and shell submodules (see Section 5), which are listed in *essential_scripts* python list. All the modules and submodules are in the *src/* directory. What follows is the detailed description of CLIO's workflow including defining the main namelist that controls CLIO.

4.1 Launch script

CLIO_start.py contains the main configuration namelist, whose content is written to **config.json** . An example of 'config' is given below:

```
config = {
    "workdir": "/path/to/your/CLIO/installation/",
    "expname": "abt3",
    "IFS_path": "/path/to/OpenIFS/initial/files/",
    "grid_type": "TL639",
    "cmip_path": "/path/to/CMIP6/data/",
    "model": "EC-Earth3",
    "variant_lab": "r1i1p1f1",
    "exp_cmip": "historical",
    "date_yr": "2008",
    "date_mo": "1",
    "date_dd": "9",
    "date_hr": "12",
    "CL_length":"15",
    "if_CMIP_in_IFS_model_level":False,
    "use_linear_hybrid2hybrid":False,
    "ML_input":  "/path/to/your/model/level/definitions",
    "ML_input_sep": ",",
```

```
    "in_HPC":"false",
    "ERA5_source":False,
    "update_SH":True,
    "update_GG":False,
    "update_UA":True,
    "update_CL":False
}
```

- **workdir** [**str**]: The run directory that should contain the running script CLIO_start.py. For TL639 resolution with 91 levels, there has to be ∼11 GB space for running CLIO smoothly. E.g. */path/to/your/CLIO/installation/*.
- **expname** [**str**]: The 4-character experiment name of the OpenIFS initial files that will be rewritten. For example, in case of ICMSHabt3INIT 'abt3' will be the expname.
- **IFS_path** [**str**]: The directory that contains the OpenIFS initial files, which CLIO will rewrite. These files are ICMSHxxxxINIT, ICMGGxxxxINIT, ICMCLxxxxINIT, ICMGGxxxxINIUA. Herer xxxx is equivalent *expname* defined above. The following horizontal resolutions with 91 model levels are available to download from the external dataset (https://doi.org/10.5281/zenodo.23364583): TL1279, TL639, TL399, TL255, TL159, TCo319, TCo639 and TCo1279. With 137 levels the following options are available from the Zenodo dataset: TL639L137 and TL1279L137.
- **grid_type** [**str**]: The spectral resolution OpenIFS will use. Currently, TL1279, TL639, TL399, TL255 and TL159 resolutions are available equivalent to N640 (16 km), N320 (31 km), N200 (50 km), N128 (78 km) and N80 (125 km) Gaussian grids, respectively. Cubic octahedral grids TCo1279, TCo639, TCo319 equivalent to grid spacing of 8 km, 16 km and 31 km are also available.
- **cmip_path** [**str**]: Path to your CMIP6 input data.
- **model** [**str**]: The standard name of the CMIP6 model, which can be found in the names of the raw CMIP6 data files downloaded from the ESGF nodes (https://esgf.github.io/nodes.html). No renaming is assumed. Model names can be found in e.g. https://pcmdi.llnl.gov/CMIP6/ArchiveStatistics/esgf_data_holdings/.
- **variant_lab** [**str**]: The variant label (i.e. the ensemble member) of the CMIP6 input data.
- **exp_cmip** [**str**]: The CMIP6 experiment type, e.g. ssp585, historical, modelSST-futArcSIC, etc. See experiment types for the ScenarioMIP: https://pcmdi.llnl.gov/CMIP6/ArchiveStatistics/esgf_data_holdings/ScenarioMIP/index.html.
- **date_yr** [**str**]: The year of your CMIP6 data.
- **date_mo** [**str**]: The month of your CMIP6 data.
- **date_dd** [**str**]: The day of your CMIP6 data.

- **date_hr** [**str**]: The hour of your CMIP6 data. Together with *date_yr*, *date_dd* and *date_hr* gives the exact date CLIO will use to update the OpenIFS initial files.
- **CL_length** [**str**]: Number of date entries in ICMCLxxxxINIT file.
- **if_CMIP_in_IFS_model_level** [**bool**]: set to **True** when the CMIP6 input data are on the same model levels as OpenIFS (either 91 or 137). This is a very special case and only applies to certain versions of EC-Earth data and ERA5 model level data, therefore in the vast majority of the cases, this should be set to **False**.
- **use_linear_hybrid2hybrid** [**bool**]: Only matters when *if_CMIP_in_IFS_model_level* is set to **False**. This variable forces CLIO to use the input hybrid to OpenIFS hybrid model level vertical interpolation scheme. To ensure that this module works properly, you need to have the model level definitions of your input data available for CLIO to read (details follow). Your input data also needs to be on hybrid levels (https://doi.org/10.1175/1520-0493(1981)109<0758:AEAAMC>2.0.CO;2). If this parameter is set to 'False' and *if_CMIP_in_IFS_model_level* is also 'False', then CLIO will apply the pressure level to OpenIFS hybrid model level interpolation scheme. This means that CLIO expects that your input data is on pressure levels.
- **ML_input** [**str**]: path of the csv file containing the model level definitions of your input data. The file should contain the level number (n), and the a and b parameters from top to bottom. An example file for MPI-ESM L95 can be found in *app/test/ECHAM_ml95_definitions.csv*.
- **ML_input_sep** [**str**]: the separator of the data in your model level csv file.
- **in_HPC** [**str**]: set to *true* (no capitals) when you run CLIO inside the container (recommended). In case you run CLIO in your custom environment, this should be $false$ (no capitals).
- **ERA5_source** [**bool**]: set to **True** when you supply ERA5 data input for CLIO.
- **update_SH** [**bool**]: set to **True** when you want to update the spectral prognostic variables in the ICMSHxxxxINIT file. If set to **False**, then module *mod_mainVARS* is switched off.
- **update_GG** [**bool**]: set to **True** when you want to update the surface variables in the ICMGGxxxxINIT file. If set to **False**, then the corresponding computations are switched off.
- **update_UA** [**bool**]: set to **True** when you want to update the 3d gridpoint variables in the ICMGGxxxxINIUA file. If set to **False**, then module *mod_UpperAtmosVARS* is switched off.
- **update_CL** [**bool**]: set to **True** when you want to update the 2d variables in the ICMCLxxxxINIT file. If set to **False**, then the corresponding computations are switched off. If set to **False** together with **update_GG**, then the entire module *mod_extraVARS* is switched off.

4.2 Variables to be updated by CLIO

At least the following variables **MUST** be downloaded from the CMIP6 model/ERA5 to be updated by CLIO. They are defined in a python list named *essential_cmip_vars*. If CLIO cannot find them in the correct directory, the user gets an error message, "Error: Missing essential CMIP variables".

- U-wind (CMIP6 convention: *ua*)
- V-wind (CMIP6 convention: *va*)
- surface pressure (CMIP6 convention: *ps*)
- air temperature (CMIP6 convention: *ta*)
- specific humidity (CMIP6 convention: *hus*)
- sea surface temperature (CMIP6 convention: *tos*)

Important note: Variables $\{ua, va, ta, hus\}$ must be either on standard hybrid levels (https://doi.org/10.1175/1520-0493(1981)109<0758:AEAAMC>2.0.CO;2) or on pressure levels. If the data is e.g. on sigma levels, the data MUST be converted to pressure/hybrid levels before supplying them to CLIO. The same applies to other non-compulsory 3d input fields such as *cli* and *clw* (see below).

The following variables can be added for CLIO to update the initial files:

- sea ice content (CMIP6 convention: *siconca*)
- temperature of soil (CMIP6 convention: *tsl*)
- sea ice temperature (CMIP6 convention: *sitemptop*)
- soil moisture content (CMIP6 convention: *mrsos*)
- surface snow amount (CMIP6 convention: *snw*)
- snow internal temperature (CMIP6 convention: *tsn*)
- skin temperature (CMIP6 convention: *ts*)
- mass fraction of cloud liquid water (CMIP6 convention: *clw*)
- mass fraction of cloud ice (CMIP6 convention: *cli*)

CLIO_start.py will search for all possible fields. The search path is given by *cmip_path*. The search string for the data file is the following: var_*model_*exp_cmip_*variant_lab_*date_yr*.nc, where *var* is the CMIP6 variable name (e.g. *ua*). If they are not found under the correct path the program can still continue running provided that all essential variables are found. Instead of CMIP6, ERA5 data can also be used to run CLIO and update OpenIFS initial files. From ERA5 both pressure and model level data are available, the latter on 137 model levels, which are equivalent to those that OpenIFS uses. When using ERA5 data, the recommended directory structure and the CMIP6 naming convention should be used in order that CLIO finds the actual data.

4.3 Updating the spectral prognostic variables

Module **mod_mainVARS.py** reads the {`ua, va, ta, ps`} CMIP6 fields (provided as NetCDF files) and remaps them onto the horizontal grid of the OpenIFS input files using CDO's bilinear remapping routine. In case namelist variable *if_CMIP_in_IFS_model_level* is set to 'False' CLIO performs vertical interpolation, either hybrid to hybrid levels or pressure to hybrid levels (see above). The user should be aware that both vertical interpolation routines use linear interpolation without considering mass conservation constraints. This is only a limitation for the specific humidity field (see Section 4.5), especially when the difference between the number of levels of the input and the output is large. After the interpolation, as part of a sanity check if enabled by the user, the remapped CMIP6 data can be plotted against the data of the original OpenIFS initial files, which is transformed into gridpoint space. The plotting option is set inside **mod_mainVARS.py**, which is controlled by parameter *check_interpolation*. Following the sanity check, all remapped fields are converted to spectral space using CDO's gp2sp functions, and all files are converted to GRIB (edition 1) format. Note that the *ua* and the *va* fields are converted to vorticity-divergence using CDO's uv2dv function.

After this, the spectral filter module, if enabled, dampens small-scale noise for variables {`vo, d, t`} as well as creates a hybrid *lnsp* file by combining large-scale CMIP6 surface pressure data with the small-scale surface pressure data read from the auxiliary files (ERA5 data). In its current form, even when the spectral filter module is disabled, CLIO still updates the *lnsp* spectral field with the hybrid spectral coefficients. Here the user can control the cut-off zonal wavenumber for the spectral filter (named as *cutoff_wavenumber*) and the zonal wavenumber limit for the large-scale CMIP surface pressure data (named as *lnsp_wavenumber*). The latter one is by default set to 36. Using a zonal wavenumber larger than 85 is not tested. Variable *cutoff_wavenumber* is by default set to 255, which should be modified based on the chosen OpenIFS resolution (i.e. it should be smaller than the truncation limit) and the input CMIP6 data. These variables can be set within **mod_mainVARS.py** .

As the last step CLIO calls **mod_outputOIFS.py** submodule, which has functions that update the GRIB files containing the initial conditions of OpenIFS. Most operations are done here with the metview-python (5.22) package. Here ICMSHxxxxINIT file is updated and a copy is saved as ICMSHxxxxINITmod into *IFS_path*.

4.4 Updating the gridpoint surface variables

4.4.1 Updating the ICMCLxxxINIT initial file

Module **mod_extraVARS.py** first updates the climatological input file ICMCLxxxxINIT, specifically the soil temperature (*stl*1), the sea ice content (*ci*), the albedo (*al*) and the leaf-area index for low and high vegetation (*lai_lv* and *lai_hv*) fields. This file contains more than one timestep (i.e. days), which determines for how long the simulation can be run. The *stl*1 field is updated with the CMIP6 *tos* field, since the SSTs in OpenIFS are read from here. The *ci* field is updated from the CMIP6 *siconca* field. If the CMIP6 input files do not

temporally overlap with the ICMCLxxxxINIT file, the nearest available CMIP6 timestep is used. An error occurs if the timestep of the ICMCLxxxxINIT file (*OIFS_CL_increment*) is set incorrectly; this value is 24 hours by default and must not be modified. The *al*, *lai_lv* and *lai_hv* fields are updated from the 30-year (1981-2010) ERA5 monthly mean values. This also applies to the *ci* field if CLIO cannot find a corresponding input file.

4.4.2 Updating the ICMGGxxxINIT initial file

After updating the ICMCLxxxxINIT file, module **mod_extraVARS.py** first reads the {`tos`} CMIP6 field (provided as NetCDF). Additionally, if the user provides other fields inside cmip_path CLIO will read these files too, which can be {`siconca, tsl, sitemptop, mrsos, snw, tsn`}. This is followed by remapping the data onto the desired horizontal grid. CLIO also creates NetCDF files from some of the fields in ICMGG...INIT and reads these files.

Updating sea surface temperature and sea ice content CLIO converts *tos*, which is in °C, to K. Variable *siconca*, which is in %, is converted to fall in between 0-1. If *siconca* is not available, ERA5 30-yr (1981-2010) monthly mean values are assigned taken from aux/. The month is selected based on namelist variable *date_mo*.

Updating sea ice temperature OpenIFS employs a four-layer sea ice scheme and therefore requires sea ice temperatures for all four layers. CMIP6, however, provides only the single-layer variable *sitemptop*. Consequently, CLIO assigns sea ice temperatures to all four layers based on this field.

If *sitemptop* is available, its values are used directly for the top ice layer (*istl*1). For the second layer below the surface (*istl*2), CLIO increases *sitemptop* by 0.6 K. This modified value is retained only where it does not exceed 271.45K (i.e. the temperature of frozen water); otherwise, the original *sitemptop* value is used for *istl*2. For the third and fourth layer (*istl*3 and *istl*4), *sitemptop* is increased by 2.6 K and 4.6 K, respectively, but CLIO retains this modified value only where the result is $\leq$ 271.45 K. These prescribed differences between the layers are computed from 30-year means (1981-2010) using ERA5 *istl* data. Note that *istl* data is only updated when *siconca* is available, otherwise the above mentioned climatological ERA5 data are used.

If *sitemptop* is not provided, CLIO assigns ERA5 30-yr (1981-2010) monthly mean values for all four ice layers, which are stored in the aux/ directory. Note that CLIO's sea ice temperature assignment is experimental and serves only to provide realistic vertical temperature distributions for the four ice layers in OpenIFS. For short-range weather forecasts, the impact of sea ice temperature on large-scale atmospheric dynamics can be assumed to be negligible. However, users should be aware of the limitations of this module in the context of their specific research objectives.

Updating soil temperature CLIO uses the CMIP6 variable *tsl* to update all four soil layers, as the differences in temperature among the OpenIFS soil layers

are small. The same approach is applied to soil moisture: the CMIP6 variable *mrsos* is assigned uniformly to all four soil layers. The variable *mrsos* is provided in units of kg m^{-2}, corresponding to the mass of water in all phases within the upper 10 cm of the soil layer. For use in OpenIFS, *mrsos* is converted to volumetric soil moisture (m^3 m^{-3}). This conversion requires the soil bulk density, which is approximated as 1300 kg m^{-3}, which is realistic regarding European soil data https://esdac.jrc.ec.europa.eu/themes/soil-bulk-density-assessments. If *tsl* and *mrsos* is not available, ERA5 30-yr (1981-2010) monthly mean values are assigned.

Updating snow fields Updating the snow-related OpenIFS fields (*sd*, snow depth; *tsn*, temperature of snow layer) requires the existence of *snw* CMIP6 data. The OpenIFS *sd* field is updated with the CMIP6 *snw* field. Regarding *tsn*, only the top snow layer is updated with CMIP6 data. If *tsn* is not provided, however *snw* is present, CLIO will assign 254.3 K (i.e. the 1981-2010 global mean value based on ERA5 data) for the top-layer snow temperature, where CMIP6 snow is present. If *snw* is not available, ERA5 30-yr (1981-2010) monthly mean values are assigned.

Updating skin temperature The skin temperature (*skt*) is updated from the CMIP6 *ts* field. If *ts* is not available, CLIO approximates *skt* by combining the CMIP6 *tos*, *sitemptop* and *tsl* fields. If *sitemptop* and *tsl* are not available, CLIO assigns ERA5 30-yr (1981-2010) monthly mean values.

Updating additional variables with ERA5 climatology In the ICMGGxxxxINIT file, there are other variables, which change in time, but are not available from CMIP6. These are the following (using ECMWF naming convention, https://codes.ecmwf.int/grib/param-db/):

- *asn*, snow albedo
- *rsn*, snow density
- *src*, skin reservoir content
- *lmlt*, lake mix-layer temperature
- *lmld*, lake mix-layer depth
- *lblt*, lake bottom temperature
- *ltlt*, lake total layer temperature
- *lshf*, lake shape factor
- *lict*, lake ice surface temperature
- *licd*, lake ice total depth
- *chnk*, Charnock
- *lai_lv*, leaf-area index, low vegetation
- *lai_hv*, leaf-area index, high vegetation

- *al*, albedo

All of these fields are updated from ERA5 30-year (1981-2010) monthly mean climatologies available from *aux/*. In case of *al*, the monthly mean ERA5 forecast albedo is used for the update.

After assigning the new fields, there is the possibility for a sanity check in case *check_surface_update* is set to 'True'. This module creates a few plots comparing some selected fields with the original and the updated gridded data. Following the sanity check, all remapped fields are converted to GRIB format. As the last step **mod_outoutOIFS.py** submodule is called, which updates the ICMGGxxxxINIT file and saves a copy as ICMGGxxxxINITmod, which can be found under *IFS_path*.

4.5 Updating the gridpoint upper atmosphere variables

Module **mod_UpperAtmosVARS.py** updates the moisture-related OpenIFS variables {`q, clwc, ciwc, cc, crwc, cswc`}, for details see `https://codes.ecmwf.int/grib/param-db/`. Here only variable q or in CMIP6 convention *hus* is required for CLIO (in NetCDF format). Optionally, the user can provide *clwc* and *ciwc* or in CMIP6 convention *clw* and *cli*, respectively. The fields that are not provided due to data limit of CMIP6 are the following: {`cc, crwc, cswc`}. These fields are assigned with zeroes implying no initial cloud fields (except for ERA5 input, see below). Since q is always updated from CMIP6, moisture will be present in the model and cloud formation will kick off instantly. Preliminary tests have indicated that realistic cloud fields have developed after about 8-12 hours of integration.

The fields that are provided for this module (*hus* and others) are remapped to the desired horizontal grid. In case *if_CMIP_in_IFS_model_level* is set to 'False' CLIO performs vertical interpolation, either hybrid to hybrid levels or pressure to hybrid levels, similarly as in **mod_mainVARS.py**. For a sanity check, q from the OpenIFS initial files and *hus* from CMIP6 are compared against each other, which can be turned off by setting *check_interpolation* to 'False'. Following the sanity check, all remapped fields are converted to GRIB format. As the last step **mod_outoutOIFS.py** submodule is called, which updates the ICMGG...INIUA file and saves a copy as ICMGG...INIUAmod, which can be found under *IFS_path*.

4.6 Special case: Updating variables using ERA5 data

The initial conditions of OpenIFS can also be updated with ERA5 data either obtained on pressure or on 137 model levels. However, certain pre-processing steps should be carried out before supplying the ERA5 fields to CLIO. First, all ERA5 fields should be split to one variable per file, where the variable name follows the CMIP6 naming convention. When pressure level data is supplied to CLIO, make sure that the pressure levels are in Pa, the dimension name is 'plev' and the longname is 'pressure'. All of these can be set with CDO's *setzaxis* function. Even though vorticity and divergence is available from ERA5, CLIO needs the u and v-wind data. However, CLIO can be supplied with all the upper air gridpoint fields (i.e., specific humidity q, specific cloud liquid water content

clwc, specific cloud ice water content *ciwc*, fraction of cloud cover *cc*, specific rain water content *crwc* and specific snow water content *cswc*). The variable names CLIO searches for are the following: *hus*, *clw*, *cli*, *cc*, *crwc* and *cswc*. Note that CLIO can only update those variables which are defined in Section 4.2 plus the additional three upper-atmosphere variables {`cc, crwc, cswc`}. To enable ERA5 input, set *ERA5_source* in the main namelist to 'True'. For the accuracy of the vertical interpolation, it is recommended to use as many pressure levels as possible.

4.6.1 Downloading ERA5 data

ERA5 data can be accessed both on pressure and on 137 model levels. These 137 model levels are the same that is used in OpenIFS. When downloading pressure level data, the user is recommended to use all available levels. Downloading ERA5 data can be obtained via the CDS web interface or via CDS API, for details visit `https://confluence.ecmwf.int/spaces/CKB/pages/129135000/How+to+download+ERA5+from+the+Climate+Data+Store+CDS`. Note that model level data cannot be obtained from the CDS web interface. It is recommended to download each variable separately so that they are stored in separate files. Make sure to select NetCDF as the download format. Note that CLIO cannot use vorticity and divergence as input; instead, download the u- and v-wind components. If model level data is downloaded, set *if_CMIP_in_IFS_model_level* to 'True'. When pressure level data is downloaded set both *if_CMIP_in_IFS_model_level* and *use_linear_hybrid2hybrid* to 'False' when running CLIO.

5 CLIO submodules

In src/ further python and shell submodules are found:

- **mod_horInterp.bash** is responsible for the horizontal interpolation of 3D atmospheric data by using CDO's bilinear remapping function.
- **mod_horInterp_2d.bash** is responsible for the horizontal interpolation of 2D surface data by using CDO's bilinear remapping function.
- **mod_inputOIFS_prepare.bash** extracts meteorological fields from the OpenIFS initial ICMSHxxxxINIT file and generates grid point NetCDF files, which CLIO uses to produce plots during the sanity check, as well as reads their metadata.
- **mod_inputOIFS_extra_prepare.bash** extracts surface fields from the OpenIFS initial ICMCLxxxxINIT and ICMGGxxxxINIT files and generates NetCDF files, which CLIO uses to produce plots during the sanity check, as well as reads their metadata.
- **mod_inputOIFS_UA_prepare.bash** extracts meteorological fields from the OpenIFS initial ICMGGxxxxINIUA file and generates NetCDF files, which CLIO uses to produce plots during the sanity check, as well as reads their metadata.
- **mod_prepICMCL.bash** creates GRIB files from the NetCDF data containing the interpolated CMIP6 fields, which are readable by metview-python and will be used to update the fields in the ICMCLxxxxINIT file.
- **mod_prepICMGG.bash** creates GRIB files from the NetCDF data containing the interpolated CMIP6 fields, which are readable by metview-python and will be used to update the fields in the ICMGGxxxxINIT file.
- **mod_prepICMUA.bash** creates GRIB files from the NetCDF data containing the interpolated 3d upper-air CMIP6 fields, which are readable by metview-python and will be used to update the fields in the ICMGGxxxxINIUA file.
- **mod_prepICMSH_step1.bash** creates the spectral *lnsp* GRIB file from the NetCDF CMIP6 data, which will be used to update the *lnsp* field in the ICMSHxxxxINIT file. This module also changes the variable code and name of the NetCDF file containing the interpolated CMIP6 U and V-wind and the air temperature fields. In addition, the module performs spectral transform on the CMIP6 air temperature data.
- **mod_prepICMSH_step1-TCO.bash** contains essentially the same commands as mod_prepICMSH_step1.bash, but used only for TCO grids.
- **mod_prepICMSH_step2.bash** generates spectral vorticity and divergence NetCDF data from the CMIP6 U and V winds.

- **mod_prepICMSH_step3.bash** generates GRIB files containing the spectral vorticity, divergence and air temperature CMIP6 fields which are readable by metview-python and will be used to update the fields in the ICMSHxxxxINIT file.
- **mod_prepERA5-clim.bash** remaps the ERA5 climatological mean fields and generates GRIB files, which are readable by metview-python and will be used to update some fields in the ICMGGxxxxINIT and the ICMCLxxxxINIT file.
- **mod_outputOIFS.py** contains the functions used for updating the ICM* initial files with the CMIP6 fields coverted to GRIB format. Here metview-python is used for the data swap.
- **mod_simple_plots.py** contains some plotting functions, particularly for the sanity check.
- **mod_spectral_filter.py** contains functions for filtering small-scale spectral noise and for constructing the hybrid *lnsp* field by adding large-scale CMIP6 and small-scale ERA5 climatological spectral data.
- **mod_vert_interpol.py** contains functions for hybrid to hybrid level, and pressure to hybrid level vertical interpolation.

6 Important notes

- If there are more than one file for a single variable within *cmip_path* CLIO will produce an error message.
- In order for CLIO to find all the CMIP6 files used for updating the OpenIFS initial conditions, all the file names should include the variable name, the model, the variant label, the experiment type and the year, which should be consistent with the variable declarations in 'configs' in **CLIO_start.py**.
- CLIO supports only NetCDF input data, the OpenIFS input data are always GRIB files, which are provided.
- If the user wants to run OpenIFS in the same resolution as the CMIP6 input data or in a lower resolution, it is recommended to turn off the spectral filter module, i.e. set *sp_filter_ON* to 'False'.
- The user should check whether the input 2d fields (i.e. *tsl*) have only the following dimensions: time, latitude and longitude. It is assumed that there is no depth dimension.
- Updating ICMGGxxxxINIUA files needs gridpoint surface pressure data for the vertical interpolation (if enabled), thus it should follow the update of ICMSHxxxxINIT file, otherwise CLIO will report that it did not find the **temp_ps_seldate_orig.nc** file.
- Since the cloud-related fields cannot be updated from CMIP6, OpenIFS needs a short spin-up period to obtain realistic precipitation and cloud fields. Typically, a minimum of 12 hrs.

- CLIO can be run on a PC instead of the container. For this the user needs CDO and ecCodes installation which can be found in $PATH, as well as the necessary python packages described in **reqs-python.txt**.
- CLIO does not change the time dimension of the initial files even though the user always specifies the time in the namelist. This means that your OpenIFS output files will start from the time specified in the original OpenIFS input files.
- Requesting the TCo1279 grid requires large memory (∼50 GB). This option was not tested by the developers.

7 Running CLIO

CLIO can be run either inside the container or as a desktop application provided that all the necessary modules and python packages are available discussed in Section 3. We recommend running the container as an interactive shell with all necessary paths bound, for example:

```
singularity shell --bind /path1/,/path2/ ./clio-image.simg
```

When generating high-resolution OpenIFS input files, ensure that CLIO has sufficient memory available. For example, generating TL639L137 files requires approximately 12 GB of memory. Note that the raw CMIP6 NetCDF files are typically large because they contain data for multiple dates. Before supplying these files to CLIO, it is recommended to split them into shorter time segments; otherwise memory issues may occur.

To launch CLIO type the following:

```
python3 CLIO_start.py
```

When CLIO is finished successfully you will see the following message printed on your screen:

```
++++++++++++++++++++++++
CLIO run successfully
++++++++++++++++++++++++
```

7.1 Testing CLIO

After downloading CLIO, the user has option to test the application. To launch CLIO's test module the user should run **CLIO_test.py**. This script updates the initial files of OpenIFS inside *INPUT_GRIB/TL399* by using test data from */app/test* . Here, there is test data for parameters {`ua, va, ta, ps, hus, tos, siconca, tsl`}, which are taken from CMIP6 (MPI-ESM1-2-HR and EC_Earth3) and manipulated to make easy-to-understand test cases. The workflow of **CLIO_test.py** follows the same logic as **CLIO_start.py**, however, there are additional post-processing steps before running the test cases. The post-processing involves creating NetCDF files from the updated OpenIFS initial condition files, which is followed by the 16 tests. At the end of the procedure CLIO will print the following on screen in case all tests are passed:

```
Tests passed: 16 out of 16
++++++++++++++++++++++++++++++++
++++  CLIO testing finished ++++
++++++++++++++++++++++++++++++++
```

To run the testing module of CLIO the user should type the following (assuming being inside of the container's interactive shell):

```
python3 CLIO_test.py path true
```

Here *path* is the location of the user's CLIO installation up to */clio_release/* directory, e.g. `/scratch/project_xxx/$USER/CLIO_installation/clio_release/`. Inside *path* there should be the user's *app/*, *INPUT_GRIB* and *container/* directory. The second input concerns whether CLIO is run inside the container or on a local PC, for the former one this is always 'true'.